Learning When to Trust Retrieval: A Disagreement-Aware Gate for Few-Shot Inverse Dynamics

Constantin Ertel Phillip Kerger
University of California, Berkeley
Preprint | 24 October 2026

Abstract. Video-based robot policies increasingly split control in two: a video model predicts the future, and an inverse-dynamics model (IDM) turns that predicted future into actions. When a new task arrives with only five demonstrations, the IDM is the part that must adapt. Retrieving the demonstrations' actions is a natural way to adapt it, and a learned gate can mix them with the IDM's own prediction. Such a gate fails exactly when it matters: if it sees only the robot's state, it keeps trusting a source-trained prediction that is wrong for the new task. We introduce a disagreement-aware gate that also sees how confident the retrieval is and how far the two predictions disagree, and we train it on deliberately corrupted predictions so that it learns when to defer to retrieval. We froze every design choice on LIBERO-Goal and then ran once on 30 sealed tasks from three other LIBERO suites, with three frozen image encoders, five demonstrations per task and three seeds. We find that, without any training on the new task, the gate beats nearest-neighbour regression on all 30 tasks for every encoder (chunk MSE 0.378 vs 0.528 with MAE) and beats the same head without the new inputs (0.378 vs 0.425). We also find where it falls short: encoder fine-tuning reaches lower error at 7 to 17 times the compute per update, and in closed-loop simulation plain nearest-neighbour regression completes the most tasks (44% vs 38%), while lower offline error does not predict success.

Keywords: inverse dynamics; retrieval; gating; few-shot adaptation; robot manipulation; LIBERO.

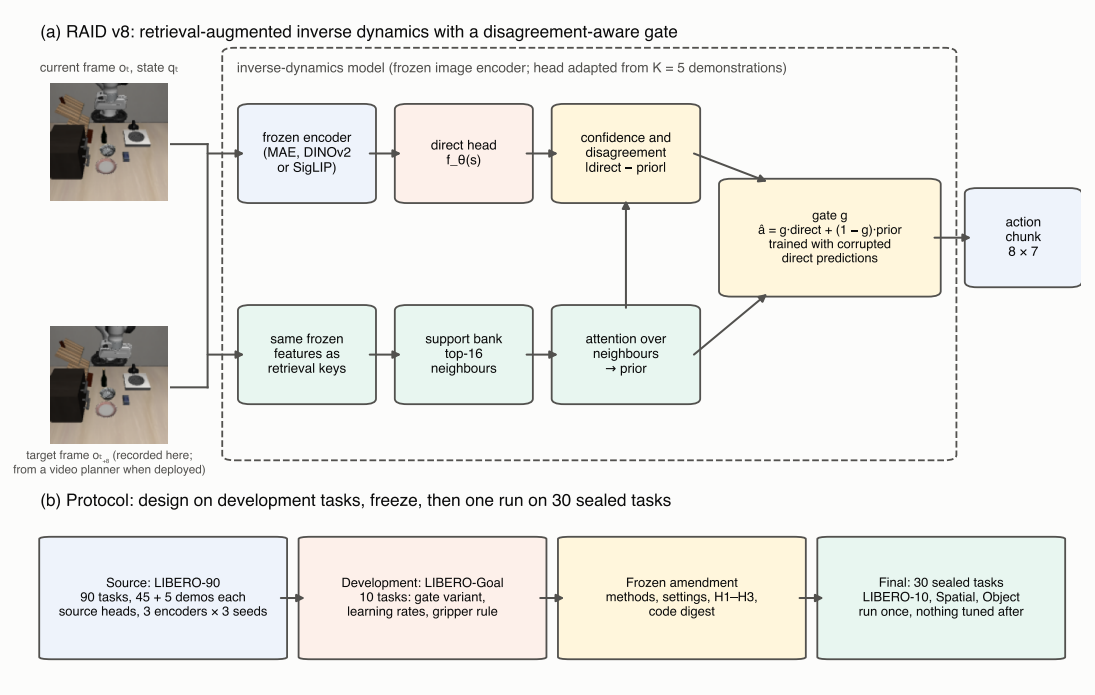


Figure 1. (a) RAID v8. A frozen encoder gives the controller state and the retrieval keys; a direct head predicts an action chunk, attention over the 16 nearest support windows forms a retrieval prior, and a gate mixes the two using the retrieval confidence and the disagreement between them. The gate is trained with corrupted direct predictions. Example frames are from a LIBERO-Goal demonstration. (b) Protocol: source heads on LIBERO-90, design on LIBERO-Goal, a frozen amendment, and one run on 30 sealed tasks.

1 Introduction

Give a robot five demonstrations of a task it has never seen, such as putting a bowl on a plate, and ask it to do the task again from a new start. Systems built on video models approach this in two steps. A generative model predicts what the scene should look like next, and an inverse-dynamics model (IDM) infers the actions that produce that change: frames in, actions out. UniPi introduced this split for text-conditioned video planning [1], and it now appears across recent systems, from visual subgoals in π0.7 [2] and the action modules of GE-Act 2.0 and VPP2 [3, 4] to the separate inverse-dynamics model of Rhoda

AI's Direct Video-Action system [5]. When the video model is fixed, adapting to a new task means adapting the IDM, and a new task rarely comes with more than a few demonstrations.

The simplest way to use those demonstrations is to look up the closest recorded moment and copy its actions, as VINN does [6]. A learned head should do better: it can attend over several retrieved candidates and blend them with its own prediction through a gate [7, 8]. In an earlier study, we found that it can also do much worse. Our attention-and-gate head (RAID v7) beat nearest-neighbour regression on average, but on one held-out LIBERO-Goal task its error was 1.7 times that of nearest-neighbour regression (0.682 vs 0.406). Its errors tracked the source-trained prediction (Pearson 0.97 across query demonstrations), not the retrieved actions. The gate saw only the robot's state. It had no way to notice that the source-trained prediction was wrong for this task.

This raises a simple question: when should a learned model trust retrieved actions over its own prediction? Our answer is that the gate making this choice must see when the two disagree. RAID v8 gives the gate two kinds of input it was missing, the confidence of the retrieval and the per-dimension disagreement between the direct prediction and the retrieved prior. The inputs alone are not enough. During source training the direct prediction is rarely far off, so the gate never learns what to do when it is. We therefore corrupt the direct prediction on purpose: for a quarter of the training rows, the mix receives another input's prediction. The gate then trains on exactly the situation it will face on a new task, a direct prediction that disagrees with confident retrieval.

We test this idea with a protocol built for statistical confidence, and we keep design and evaluation apart. Every design choice, including the gate variant, the learning rates and the closed-loop gripper rule, was made on the ten LIBERO-Goal tasks. We then froze the methods, settings, analysis code and three hypotheses in a digest-bound amendment and ran the benchmark once on 30 sealed tasks from three other LIBERO suites [9], with three image encoders: MAE [10], DINOv2 [11] and SigLIP [12]. Every comparison uses paired task-level bootstrap intervals and Holm-corrected tests.

Our contributions:

1. **A disagreement-aware retrieval gate, trained with direct-path corruption.** Without target training, it has lower action error than nearest-neighbour regression on all 30 sealed tasks for each of three encoders, and lower error than the same head without the new inputs (Section 6.1). The corruption is essential: on development tasks, the new inputs without it make the head worse, not better (Section 6.2).
2. **A compute-aware comparison with fine-tuning.** A head update on frozen features costs 4 to 7 ms; a LoRA or full fine-tuning update costs 52 or 128 ms. The gated head reaches its best error within 25 updates, but encoder fine-tuning reaches lower error with more compute (Section 6.3).
3. **A closed-loop test without a video generator,** which replays recorded target frames in the LIBERO simulator. The adapted gated head completes more tasks than head-only fine-tuning, LoRA and full fine-tuning; nearest-neighbour regression still completes the most, and lower offline error does not predict success (Sections 6.4 and 6.5).

Figure 1 summarizes the model and the protocol.

2 Related work

Video planning with inverse dynamics. UniPi trains a separate inverse model to execute generated videos [1]. SuSIE proposes subgoal images with an image-editing model and reaches them with a goal-conditioned low-level policy [13], and AVDC recovers actions from generated videos through dense correspondences [14]. π0.7 conditions a generalist policy on visual subgoals [2], GE-Act 2.0 and VPP2 pair a visual predictor with an action module [3, 4], and Rhoda AI's Direct Video-Action model converts predicted video into robot actions with a separate inverse-dynamics model [5]. Predictive inverse-dynamics models scale this idea [15]. In all of these systems, the component that maps a predicted future to actions must work on each new task. We keep the planner out of the evaluation and study only how this component adapts from five demonstrations.

Inverse dynamics with many labels. VPT trains an IDM on a small labeled set and uses it to label large amounts of online video [16], and DreamGen labels generated robot videos with an IDM [17]. The label efficiency of IDMs depends strongly on the setting [18], and in gameplay, aggregate IDM accuracy can hide failures on rare actions [19]. These works train an IDM once on many labels. We study the opposite case: the IDM is already trained, and a new task brings five labeled demonstrations.

Retrieval for action prediction. VINN predicts actions by locally weighted nearest-neighbour regression on frozen features [6]. REGENT retrieves demonstrations and blends the nearest action with a learned policy through a distance-based weight [7]. RT-Cache acts without training by retrieving stored trajectory segments [20], and ReCAP adds trajectory retrieval to a frozen policy to cover new tasks [21]. These methods fix the mixing rule or tie it to distance; our gate is learned and sees the disagreement between the two predictions.

Learned interpolation between a model and retrieval. Nearest-neighbour language models interpolate a parametric and a retrieval distribution with a fixed weight [8]. Adaptive kNN machine translation learns the number of neighbours and the interpolation weight from retrieval distances and the number of distinct retrieved values [22], and efficient kNN language models learn when retrieval helps and how much weight to give it [23]. Our retrieval-confidence inputs follow the same idea. To our knowledge, these gates do not receive the difference between the parametric and the retrieved prediction, and they are not trained on examples where the parametric prediction is wrong. For continuous actions, this difference is defined per action dimension, and our corruption training supplies those examples.

Learning when to defer. Learning to defer trains a model to hand a decision to another predictor when it is likely to be wrong [24, 25]. Our gate makes a soft, per-dimension version

of this decision between a parametric head and retrieval.

Few-shot data retrieval and lightweight adaptation. Behavior Retrieval, STRAP, FlowRetrieval and importance-weighted retrieval select prior data to train few-shot policies [26, 27],[28, 29]; HAND retrieves and then fine-tunes [30]. LoRA [31] and task-specific adapters [32] adapt large encoders cheaply, VLA-Pro retrieves and fuses task-specific adapters [33], and the fine-tuning recipe strongly changes the LIBERO success of large policies [34]. We compare against head-only, LoRA and full fine-tuning on the same support sets, tune their learning rates on development tasks, and report their compute.

Visual representations for control. R3M and VC-1 study which pretrained image encoders serve robot learning, and VC-1 finds that no single encoder is best on all tasks [35, 36]. We therefore test every hypothesis with three encoders trained with different objectives: MAE, DINOv2 and SigLIP.

Offline metrics and task success. In robomimic, the checkpoint with the lowest validation loss was often not the most successful one [37], and SIMPLER reports that validation action error is a poor proxy for real-world success [38]. Our closed-loop test checks this for few-shot inverse dynamics.

End-to-end few-shot policies. Generalist policies such as Octo and $\pi 0$ are adapted as a whole and report task success [39, 40]; in DISC's five-demonstration LIBERO-Goal evaluation, success reaches 55% for Octo and 73% for DISC [41], under a protocol different from ours. We study a component of a modular pipeline and do not compete with these systems on task success.

3 Problem setting

We isolate one question: given five demonstrations of a new task, how well can an inverse-dynamics model infer the right actions?

Inverse dynamics. At time t the IDM receives the current RGB image ot, the target image ot+$_8$, the robot state qt (7 joint and 2 gripper positions) and predicts the next 8 actions $\hat{a}$t:t+$_7$ $\in \mathbb{R}$^{8×7} in LIBERO's action space. Actions and states are normalized with statistics of the source training data.

Few-shot adaptation. A new task provides K = 5 demonstrations, the support set. A method may use them for retrieval, for parameter updates, or both. It is evaluated on 20 other demonstrations of the same task, the query set.

Offline metric (primary). Every stride-1 window of each query demonstration is evaluated with its recorded target image and recorded action chunk. The error of a window is the mean squared error over its 56 normalized action values; errors are averaged per query demonstration and then over the 20 demonstrations. We also report R^2, the mean over the 7 action dimensions of 1 − SSE/SST. Because the target image is recorded, the metric measures action inference given a correct target and does not depend on any generator.

Closed-loop metric (secondary). The simulator starts from the recorded first state of a query demonstration. Before each 8-step chunk, the current image is matched to the closest recorded frame of that demonstration (cosine distance of frozen features, searched forward over 24 frames, never backwards), and the recorded frame 8 steps later becomes the target. The predicted chunk is executed, and success is LIBERO's task predicate within 1.5 times the demonstration length plus 20 steps. This isolates the IDM from planner errors.

Compute. Adaptation GPU-seconds are the summed wall-clock time of parameter updates on one GPU, without evaluation; we also report time per update and trainable parameters. One-time costs shared by all methods (feature caching and source training) are reported separately.

4 Method

RAID v8 has three parts: a direct head that predicts actions from the current state, a retrieval path that proposes actions from the support demonstrations, and a gate that decides, for each action dimension, how much to trust each. We first describe the shared retrieval, then why the earlier gate fails, then the fix.

4.1 Retrieval-augmented inverse dynamics

A frozen image encoder φ maps each image to a 768-dimensional feature. The controller state is s = [φ(ot), φ(ot+$_8$), qt, H]. A support bank stores, for every window of the support demonstrations, the frozen features of its current and target images, its state, its normalized action chunk and its demonstration. Retrieval uses the distance

$$\begin{aligned} d = {} & 0.5\left(1 - \cos(\varphi(o_t), \varphi(o'_t))\right) \\ & + 0.5\left(1 - \cos(\varphi(o_{t+H}), \varphi(o'_{t+H}))\right) \\ & + 0.1 \cdot \mathrm{mean}\big((q_t - q'_t)^2\big), \end{aligned}$$

and returns the k = 16 closest windows. Because the encoder stays frozen in all head methods, the retrieval geometry never changes during adaptation.

4.2 Why a state-only gate fails

The RAID v7 head computes a direct prediction from an MLP trunk, an attention-weighted prior over the 16 retrieved action chunks, and a per-dimension gate g = σ(W_g s) that mixes them; an auxiliary loss pulls attention towards candidates whose actions match the target. The gate is a function of the state s only. If the source-trained trunk is wrong for a new task, nothing in s reveals it, so the gate cannot move towards the prior. This is the failure we observed.

4.3 A gate that sees disagreement

RAID v8 keeps the trunk, the attention and the selection loss of v7 and changes the gate:

$$\text{direct} = f_\theta(s), \qquad \text{prior} = \textstyle\sum_i \alpha_i a^i,$$

where αi is the attention weight of candidate i and a^i its action chunk. The gate sees a confidence vector

$$c = \left[\, \mathrm{sim}_{(1)},\ \overline{\mathrm{sim}},\ \tilde{H}(\alpha),\ |\text{direct} - \text{prior}| \,\right],$$

with the absolute disagreement taken per action element and detached from the gradient, and computes $g = \sigma(\text{MLP_g}([s, c])) \in (0, 1)^{56}$, with a 256-unit hidden layer, and $\hat{a} = g \odot \text{direct} + (1 - g) \odot \text{prior}$. Without any valid candidate the gate is fixed at 1.

4.4 Teaching the gate with wrong predictions

The new inputs only help if the gate has seen large disagreements during training. On source tasks the trunk is usually right, so we create disagreements on purpose: in each training batch, each row's direct prediction is replaced, with probability 0.25 and inside the mix only, by the detached direct prediction of a random other row. For these rows the correct action is closer to the prior, and the loss teaches the gate to take it when the disagreement is large and retrieval is confident. The trunk receives no gradient from corrupted rows. Corruption stays on during target adaptation.

4.5 Source training

One head per type, encoder and seed is trained on the 90 LIBERO-90 tasks (45 training and 5 validation demonstrations each) for 4,000 updates. Each batch draws 8 tasks; for each, an episodic bank of 2, 5 or 10 training demonstrations and 16 query windows from other training demonstrations of that task. The loss averages the retrieval-present error and the error with retrieval removed, plus 0.1 times the selection loss (AdamW, learning rate $3 \cdot 10^{-4}$, weight decay 10^{-4}, gradient clipping 1.0). The checkpoint with the lowest validation error, checked every 250 updates, is kept. With retrieval removed, the same network is a plain MLP, which serves as the starting point of head-only fine-tuning.

4.6 Target adaptation

Head methods update the whole head on frozen features with batch 64 for up to 200 updates. Retrieval-based heads retrieve from the five support demonstrations; during adaptation each support window excludes its own demonstration from the bank, and at evaluation the full bank is used.

5 Experimental protocol

5.1 Data and splits

We use the LIBERO datasets [9] from their public mirror. Source: all 90 LIBERO-90 tasks. Development: the 10 LIBERO-Goal tasks, which our earlier work had already used and which we therefore used only for design. Final: the 30 tasks of LIBERO-10, LIBERO-Spatial and LIBERO-Object, all of them, none selected; their files were downloaded and read only in the final run. LIBERO-10 shares scenes with LIBERO-90 but has different tasks; LIBERO-Spatial and LIBERO-Object use other layouts and objects. For each target task, a seeded permutation of its 50 demonstrations gives a support pool of 30 and a query set of 20, and support seeds 42, 43 and 44 each draw K = 5 support demonstrations.

5.2 Methods

Table 1 lists the compared methods. Head-only fine-tuning uses the same source network as RAID v8 with retrieval off, so the two differ only in retrieval. LoRA (rank 8, $\alpha = 16$, query-key-value and output projections of the last two MAE blocks) and full fine-tuning update the encoder and the head, are run with MAE and seed 42 only, and use retrieval off. All learning rates were chosen on development tasks from three-value grids.

5.3 Pre-registration and statistics

After development we froze the method set, all settings, the analysis code and three hypotheses in an amendment (SHA-256 digest 84375c3b…), and the final run checked this digest before reading any final file. The hypotheses, tested separately for each encoder with MAE as the primary encoder:

- **H1:** without target training, RAID v8 has lower chunk MSE than RAID v7.
- **H2:** without target training, RAID v8 has lower chunk MSE than the best training-free baseline on development tasks, which was k-NN.
- **H3:** after 25 updates, RAID v8 has lower chunk MSE than head-only fine-tuning after 25 updates.

For each task we average the paired difference over the three seeds. The 95% interval is a percentile bootstrap over the 30 tasks (10,000 resamples), the one-sided p-value is the share of bootstrap means at or above zero, and Holm's correction is applied over H1–H3. A hypothesis is supported if its Holm-adjusted p-value is below 0.05 and its interval lies entirely below zero. Everything else is descriptive.

5.4 Closed loop and the gripper rule

The closed-loop test runs 20 query episodes per final task for k-NN, RAID v7 and RAID v8 without training, head-only and RAID v8 after 25 updates, and LoRA and full fine-tuning after 200 updates (seed 42, MAE). On development tasks we saw that learned heads output gripper commands between open and closed, which often fails to grasp; rounding the gripper command to fully open or fully closed raised head-only success on a development pick-and-place task from 30% to 60% and k-NN from 90% to 100%. The frozen protocol therefore rounds the executed gripper command for every method. No model changes.

6 Results

6.1 Without target training, the gate beats nearest-neighbour regression on every task

H1 and H2 were supported for all three encoders (Figure 3). RAID v8 had lower error than k-NN on all 30 tasks for every encoder (MAE −0.151, 95% CI [−0.169, −0.133]; DINOv2 −0.073 [−0.088, −0.059]; SigLIP −0.084 [−0.097, −0.072]), and even its worst task was better than k-NN (Figure 2a). It also had lower error than the REGENT-style blend on all 30 tasks for every encoder. Against RAID v7 it was better on 26, 30 and 29 of 30 tasks (MAE −0.047 [−0.062, −0.034]; DINOv2 −0.099

Table 1. Methods. Trainable parameters are those updated on the target task.

Method	Retrieval	Updated on target	Learning rate	Trainable params
VINN-style k-NN (k = 4)	yes	none	—	0
REGENT-style blend	yes	none	—	0
RAID v7 head	yes	head	10^{-4}	1,420,832
RAID v8 head (ours)	yes	head	10^{-4}	1,759,728
Head-only fine-tuning	no	head	$3{\cdot}10^{-4}$	1,759,728
LoRA (MAE)	no	adapter and head	$3{\cdot}10^{-4}$	1,833,456
Full fine-tuning (MAE)	no	encoder and head	10^{-5}	87,558,384

Table 2. Final tasks, no target training: chunk MSE (mean over 30 tasks × 3 seeds) and R^2. Lower MSE and higher R^2 are better.

Method	MAE	DINOv2	SigLIP	R^2 (MAE)
Source head, retrieval off	0.683	0.655	0.763	0.05
VINN-style k-NN	0.528	0.421	0.448	0.25
REGENT-style blend	0.554	0.388	0.437	0.22
RAID v7	0.425	0.447	0.446	0.38
RAID v8 (ours)	**0.378**	**0.348**	**0.364**	**0.46**

[−0.124, −0.075]; SigLIP −0.082 [−0.100, −0.064]); on the few tasks where v7 was better, the difference was at most 0.011 (Figure 2b).

6.2 The gate needs corruption training

The development tasks contained the ablation that motivated the design (Figure 4, Appendix B). Adding the confidence and disagreement inputs without corruption training made the head worse than RAID v7 for all three encoders (MAE 0.757 vs 0.637). With corruption training, the same head was best (0.494, against 0.592 for k-NN). Without examples of a wrong direct prediction, the gate learns to trust the trunk whenever it is confident; the corrupted rows teach it the opposite case. On the development task where RAID v7 had failed in our earlier study (wine bottle on top of the cabinet), the MAE error was 1.192 for RAID v7, 0.705 for RAID v8 and 0.643 for k-NN.

6.3 With a few updates, the advantage shrinks and compute decides

The gate helps most before any update. With updates, the picture is mixed. H3 was supported for DINOv2 (−0.015 [−0.021, −0.011], 28 of 30 tasks) and SigLIP (−0.021 [−0.029, −0.015], 28 of 30) but not for MAE (+0.001 [−0.005, +0.007], 16 of 30). Two further observations are descriptive. First, after updates RAID v7 had slightly lower error than RAID v8 for all three encoders (MAE 0.323 vs 0.341 at 25 updates): the gate's preference for retrieval, which helps without training, limits what head updates can add. Second, RAID v8 reached its lowest error within 25 to 100 updates and then rose slightly, while head-only fine-tuning changed little after 100 updates.

Encoder tuning reached the lowest error: full fine-tuning 0.287 and LoRA 0.293 after 200 updates, against 0.316 for the best head method. It also costs more. One update cost 7.3 ms for the RAID v8 head, 52 ms for LoRA and 128 ms for full fine-tuning, and 200 updates took 1.3, 10.5 and 25.8 GPU-seconds (Figure 5). At 25 updates the three reached similar error (0.341, 0.335 and 0.341) after 0.17, 1.3 and 3.2 GPU-seconds. The one-time costs are shared by all methods: encoding all frames of the 30 final tasks with three encoders took 7.6 minutes, and training one source head took about 2.8 minutes on an A100.

6.4 In closed loop, the adapted head beats fine-tuning, but k-NN completes the most tasks

The disagreement-aware gate also helped in closed loop: without training, RAID v8 succeeded in 17% of episodes against 5% for RAID v7 (paired difference +0.125, 95% CI [+0.04, +0.22], better on 18 tasks and worse on 5). After 25 updates, RAID v8 succeeded in 38% of episodes against 16% for head-only fine-tuning (+0.22 [+0.10, +0.34], better on 20 tasks and worse on 4), and it was ahead of LoRA (31%) and full fine-tuning (27%) after 200 updates, although the difference to LoRA was not clear (+0.07 [−0.02, +0.17]). Plain k-NN, however, succeeded most often (44%); the difference to RAID v8 after 25 updates was −0.06 [−0.17, +0.05], better on 10 tasks and worse on 16 (Figure 6). These closed-loop comparisons were not preregistered as hypotheses.

6.5 Offline error does not predict closed-loop success

k-NN had the highest offline error of the seven closed-loop conditions (0.526, MAE, seed 42) and the highest success; LoRA and full fine-tuning had the lowest offline error (0.293 and 0.287) and lower success than the adapted RAID v8 head (offline 0.337). Across the 210 task–method pairs, the rank correlation between lower offline error and higher success was −0.06, that is, none; within a task it averaged 0.16. Copying recorded actions keeps them physically consistent, and grasping depends on a few decisive steps that an average-error metric weights like any other. Earlier imitation-learning studies report the same gap between validation loss and success [37, 38].

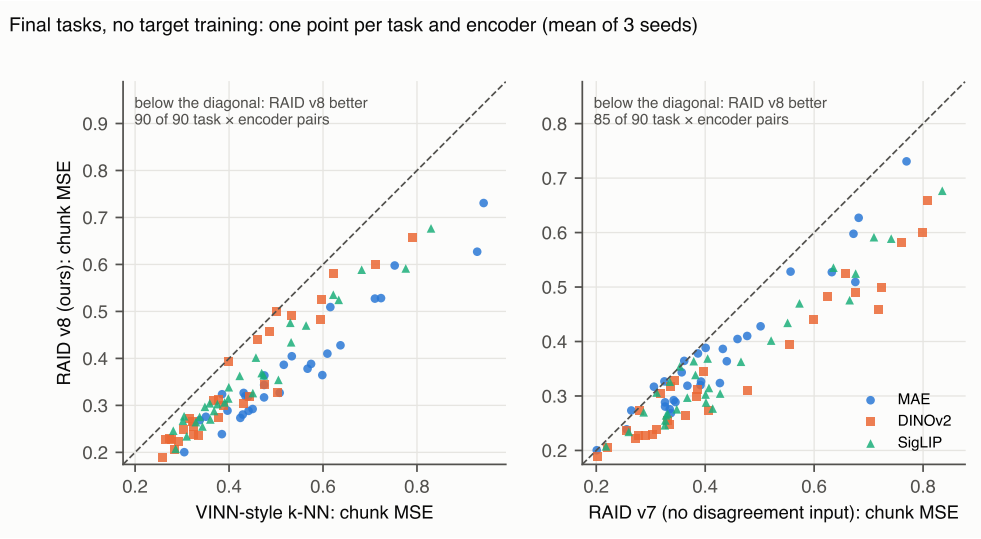


Figure 2. Final tasks, no target training: RAID v8 error for each task and encoder (mean of three seeds) against k-NN (a) and against RAID v7 (b). Points below the dashed line favour RAID v8.

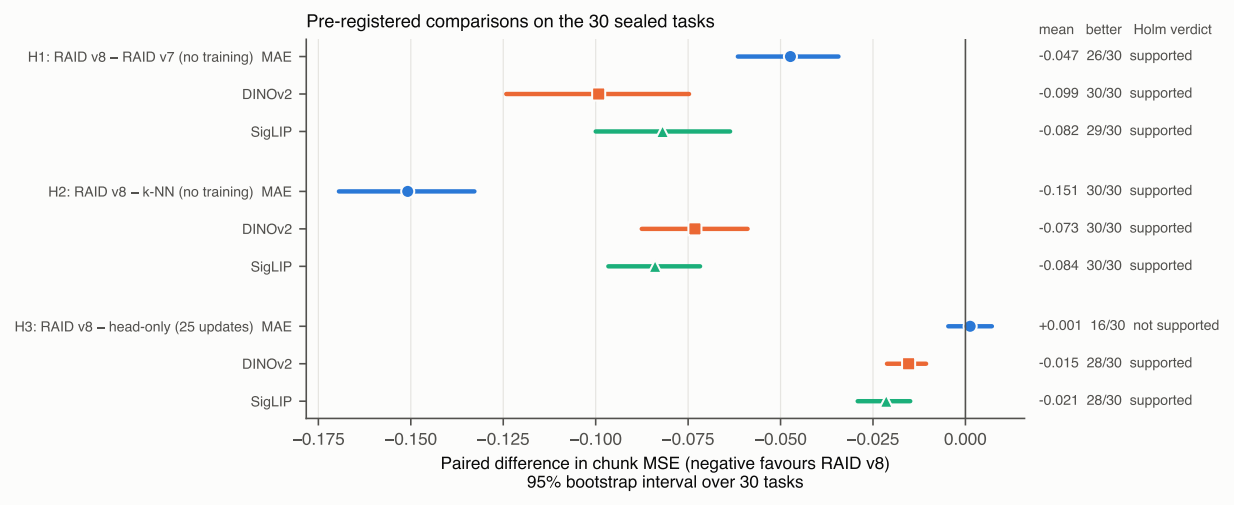


Figure 3. The three pre-registered comparisons for each encoder: paired difference in chunk MSE with a 95% bootstrap interval over the 30 tasks, the number of tasks favouring RAID v8, and the Holm-corrected verdict.

7 Discussion

When should a model trust retrieval? A learned retrieval head can be worse than the nearest neighbour it is built on, because its gate cannot tell when its own prediction is wrong. Showing the gate the disagreement, and training it on cases where the prediction is wrong, removes this failure: without any training on the target task, the head was never worse than k-NN on any of 90 task–encoder combinations. The method adds 339,000 parameters to the head and 0.4 ms per update.

What the gate does not fix. The gate improves the starting point of adaptation more than its end point. After updates, the head without the new inputs was slightly better, and fine-tuning the encoder reached the lowest error with 7 to 17 times the compute per update.

Offline error is not control. Lower action error did not mean more completed tasks. Offline error remains useful to compare inverse-dynamics models under a fixed target, but claims about control need closed-loop evidence, and the closed loop favoured copying recorded actions.

Limitations.

- All tasks are simulated LIBERO tasks with five demonstrations each, and the closed-loop test uses recorded target frames, not a video planner.
- The design was developed on LIBERO-Goal tasks that our earlier study had already used; the final tasks were never used for design.
- LoRA, full fine-tuning and the closed-loop test use one seed and one encoder (MAE).
- The gripper rounding rule was chosen on development tasks after observing closed-loop failures; it is applied identically to all methods.
- We cannot verify that the pretrained encoders never saw

Table 3. Final tasks: chunk MSE after n target updates (mean over 30 tasks × 3 seeds; encoder tuning: MAE, seed 42) and the time per update on an NVIDIA A100 80GB PCIe.

Method	Encoder	0	25	100	200	ms per update
Head-only fine-tuning	MAE	0.683	0.339	0.316	0.318	3.6
RAID v7 head	MAE	0.425	0.323	0.314	0.316	6.9
RAID v8 head (ours)	MAE	0.378	0.341	0.332	0.340	7.3
LoRA	MAE	0.693	0.335	0.300	0.293	51.9
Full fine-tuning	MAE	0.693	0.341	0.288	0.287	127.5
Head-only fine-tuning	DINOv2	0.655	0.323	0.308	0.307	—
RAID v8 head (ours)	DINOv2	0.348	0.308	0.313	0.328	—
Head-only fine-tuning	SigLIP	0.763	0.361	0.346	0.342	—
RAID v8 head (ours)	SigLIP	0.364	0.339	0.352	0.366	—

Table 4. Closed-loop success with recorded target frames (20 episodes per task, seed 42, MAE, rounded gripper), with a 95% bootstrap interval over the 30 tasks.

Method	LIBERO-10	LIBERO-Spatial	LIBERO-Object	All 30 tasks	95% CI
VINN-style k-NN	0.23	0.52	0.58	**0.44**	[0.35, 0.53]
RAID v7, no training	0.06	0.07	0.01	0.05	[0.02, 0.09]
RAID v8, no training	0.08	0.12	0.32	0.17	[0.10, 0.26]
Head-only, 25 updates	0.03	0.40	0.07	0.16	[0.08, 0.26]
RAID v8, 25 updates	0.12	0.58	0.44	0.38	[0.27, 0.50]
LoRA, 200 updates	0.15	0.57	0.21	0.31	[0.22, 0.40]
Full fine-tuning, 200 updates	0.10	0.40	0.30	0.27	[0.20, 0.33]

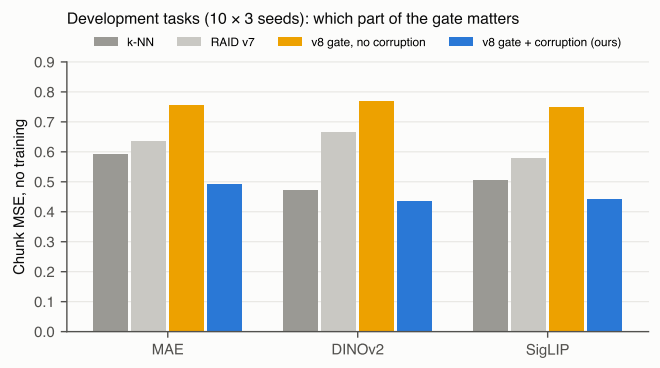


Figure 4. Development tasks, no target training: chunk MSE of k-NN, RAID v7, the v8 gate without corruption training and the v8 gate with corruption training (10 tasks × 3 seeds per encoder). These are design data, not held-out results.

LIBERO images, so we make no claim that the tasks are unseen by them.

- The bootstrap treats the 30 tasks as exchangeable; the three suites differ, and Table 4 reports them separately.

Where do we go from here? Three questions follow directly from these results. First, whether a schedule that turns corruption off during adaptation keeps both the strong start and the end point of the head without the new inputs; this needs a new frozen study. Second, why copying recorded actions completes more tasks than predictions with lower error, which calls for a closed-loop metric that weights the few decisive steps of a grasp. Third, whether the gate keeps its advantage behind a real video planner, whose predicted frames are imperfect, and on a physical robot.

8 Conclusion

A gate that mixes a learned prediction with retrieved actions needs to know when the learned prediction is wrong. Showing the gate the disagreement between the two, and training it on predictions that are wrong on purpose, made a retrieval-augmented inverse-dynamics model better than nearest-neighbour regression and better than the same head without these inputs on all 30 sealed LIBERO tasks, with three different frozen encoders and no training on the new task. The gain is largest at the start of adaptation. Encoder fine-tuning still reaches lower offline error with more compute, and in closed loop plain nearest-neighbour regression completed the most tasks. For few-shot inverse dynamics, the next step is to measure, and then optimize, what makes actions succeed rather than what makes them close.

Acknowledgments and Declarations

To be completed by the authors: funding, conflicts of interest, and data-use and licence statements for LIBERO and the pre-trained encoders. Nothing is stated here until confirmed.

Reproducibility and Artifact Availability

All numbers come from collected, SHA-256-verified run outputs, computed by paper/v8_numbers.py:

- Final run (once): artifacts/raid_v8/final_20261023/ (amendment.json with digest `84375c3be0c0ddb953a34ccfa453a15f236aa7ae65ddb6139ed87382a05ee4b8`, collection/v8_final/ with all unit files and final_results.json).
- Development run: artifacts/raid_v8/dev_20261022/ (collection/v8_dev/, including the 27 source heads and normalizers bound by the amendment).

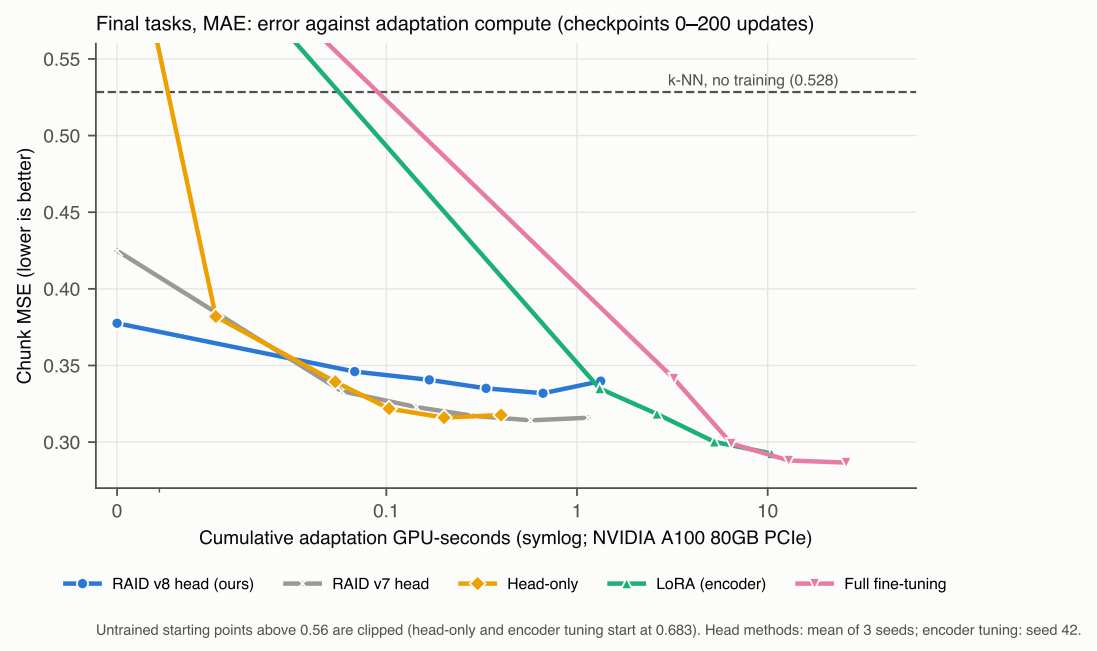


Figure 5. Final tasks, MAE: chunk MSE against cumulative adaptation GPU-seconds. RAID v8 starts lowest; encoder tuning reaches the lowest error with one to two orders of magnitude more GPU-seconds.

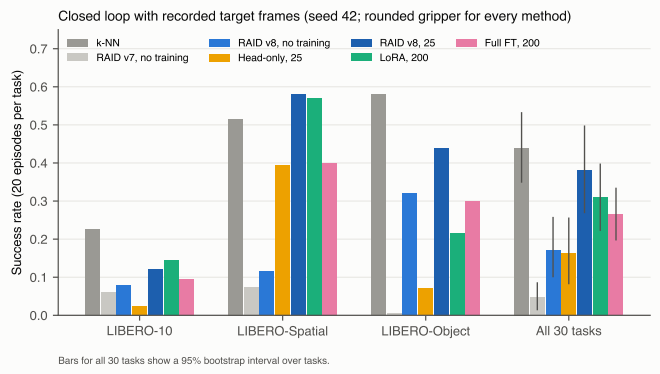


Figure 6. Closed-loop success per suite and over all 30 tasks for the seven conditions.

- Code: raid_experiments/v8/ (data splits, encoders, feature caches, heads, source training, target adaptation, closed loop, stage driver and analysis); design and hypotheses: docs/RAID_V8_PLAN.md.
- Figures: paper/figures/make_figures.py.

The data are the public LIBERO datasets; every downloaded file's SHA-256 is recorded with the run. Pretrained encoders are fixed by checkpoint hash (MAE) or Hugging Face revision (DINOv2, SigLIP).

References

[1] Du et al. Learning Universal Policies via Text-Guided Video Generation. arXiv preprint, 2023. arXiv:2302.00111.

[2] Physical Intelligence. π0.7: a Steerable Generalist Robotic Foundation Model with Emergent Capabilities. 2026. arXiv:2604.15483.

[3] AgiBot Research Team et al. GE-Act 2.0: Pretraining and Scaling a World-Action Model for Robotic Manipulation. 2026. arXiv:2609.05588.

[4] Guo et al. Video Prediction Policy 2: Predict Better, Act Better. arXiv preprint, 2026. arXiv:2610.10270.

[5] Rhoda AI Team. Causal Video Models Are Data-Efficient Robot Policy Learners. Rhoda AI research blog, 10 March 2026. rhoda.ai/research/direct-video-action.

[6] Pari et al. The Surprising Effectiveness of Representation Learning for Visual Imitation. Robotics: Science and Systems, 2022. arXiv:2112.01511.

[7] Sridhar et al. REGENT: A Retrieval-Augmented Generalist Agent That Can Act In-Context in New Environments. International Conference on Learning Representations, 2025. arXiv:2412.04759.

[8] Khandelwal et al. Generalization through Memorization: Nearest Neighbor Language Models. International Conference on Learning Representations, 2020. arXiv:1911.00172.

[9] Liu et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. Advances in Neural Information Processing Systems, Datasets and Benchmarks, 2023. arXiv:2306.03310.

[10] He et al. Masked Autoencoders Are Scalable Vision Learners. IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022. arXiv:2111.06377.

[11] Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. Transactions on Machine Learning Research, 2024. arXiv:2304.07193.

[12] Zhai et al. Sigmoid Loss for Language Image Pre-Training. IEEE/CVF International Conference on Computer Vision, 2023. arXiv:2303.15343.

[13] Black et al. Zero-Shot Robotic Manipulation with Pretrained Image-Editing Diffusion Models. International Conference on Learning Representations, 2024. arXiv:2310.10639.

[14] Ko et al. Learning to Act from Actionless Videos through Dense Correspondences. International Conference on Learning Representations, 2024. arXiv:2310.08576.

[15] Tian et al. Predictive Inverse Dynamics Models are Scalable Learners for Robotic Manipulation. arXiv preprint, 2024. arXiv:2412.15109v1.

[16] Baker et al. Video PreTraining (VPT): Learning to Act by Watching Unlabeled Online Videos. Advances in Neural Information Processing Systems, 2022. arXiv:2206.11795.

[17] Jang et al. DreamGen: Unlocking Generalization in Robot Learning through Video World Models. arXiv preprint, 2025. arXiv:2505.12705.

[18] Morin et al. On the Sample Efficiency of Inverse Dynamics Models for Semi-Supervised Imitation Learning. Proceedings of the 43rd International Conference on Machine Learning, 2026. arXiv:2602.02762.

[19] Pillai et al. Pixels to Keys: Exploring Spatial and Motion Cues in Gameplay Inverse Dynamics. arXiv preprint, 2026. arXiv:2609.37907.

[20] Kwon et al. RT-Cache: Training-Free Retrieval for Real-Time Manipulation. arXiv preprint, 2025. arXiv:2505.09040.

[21] Park et al. Retrieve, Don't Retrain: Extending Vision Language Action Models to New Tasks at Test Time (ReCAP). 2026. arXiv:2606.15631.

[22] Zheng et al. Adaptive Nearest Neighbor Machine Translation. Annual Meeting of the Association for Computational Linguistics, 2021. arXiv:2105.13022.

[23] He et al. Efficient Nearest Neighbor Language Models. Conference on Empirical Methods in Natural Language Processing, 2021. arXiv:2109.04212.

[24] Madras et al. Predict Responsibly: Improving Fairness and Accuracy by Learning to Defer. Advances in Neural Information Processing Systems, 2018. arXiv:1711.06664.

[25] Mozannar and Sontag. Consistent Estimators for Learning to Defer to an Expert. International Conference on Machine Learning, 2020. arXiv:2006.01862.

[26] Du et al. Behavior Retrieval: Few-Shot Imitation Learning by Querying Unlabeled Datasets. arXiv preprint, 2023. arXiv:2304.08742.

[27] Memmel et al. STRAP: Robot Sub-Trajectory Retrieval for Augmented Policy Learning. International Conference on Learning Representations, 2025. arXiv:2412.15182.

[28] Lin et al. FlowRetrieval: Flow-Guided Data Retrieval for Few-Shot Imitation Learning. Conference on Robot Learning, 2024. PMLR v270.

[29] Xie et al. Data Retrieval with Importance Weights for Few-Shot Imitation Learning. Conference on Robot Learning, 2025. arXiv:2509.01657.

[30] Hong et al. HAND Me the Data: Fast Robot Adaptation via Hand Path Retrieval. arXiv preprint, 2025. arXiv:2505.20455v4.

[31] Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. International Conference on Learning Representations, 2022. arXiv:2106.09685.

[32] Liu et al. TAIL: Task-specific Adapters for Imitation Learning with Large Pretrained Models. International Conference on Learning Representations, 2024. Paper.

[33] Si et al. VLA-Pro: Cross-Task Procedural Memory Transfer for Vision-Language-Action Models. arXiv preprint, 2026. arXiv:2605.29562v1.

[34] Kim et al. Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success. arXiv preprint, 2025. arXiv:2502.19645.

[35] Nair et al. R3M: A Universal Visual Representation for Robot Manipulation. Conference on Robot Learning, 2022. arXiv:2203.12601.

[36] Majumdar et al. Where are we in the search for an Artificial Visual Cortex for Embodied Intelligence? Advances in Neural Information Processing Systems, 2023. arXiv:2303.18240.

[37] Mandlekar et al. What Matters in Learning from Offline Human Demonstrations for Robot Manipulation. Conference on Robot Learning, 2021. arXiv:2108.03298.

[38] Li et al. Evaluating Real-World Robot Manipulation Policies in Simulation. Conference on Robot Learning, 2024. arXiv:2405.05941.

[39] Octo Model Team et al. Octo: An Open-Source Generalist Robot Policy. arXiv preprint, 2024. arXiv:2405.12213.

[40] Black et al. π0: A Vision-Language-Action Flow Model for General Robot Control. arXiv preprint, 2024. arXiv:2410.24164.

[41] Ren et al. DISC: Decoupling Instruction from State-Conditioned Control via Policy Generation. arXiv preprint, 2026. arXiv:2605.20856.

A Settings

Table A1. Settings shared by all methods unless stated.

Item	Value
Encoders	MAE ViT-B/16 (normalized CLS), DINOv2 ViT-B/14 (facebook/dinov2-base), SigLIP ViT-B/16 (google/siglip-base-patch16-224); all frozen, 768-d, 224 × 224 bicubic input
State	7 joint + 2 gripper positions; actions 8 × 7; both normalized with LIBERO-90 training statistics
Head trunk	MLP 512–256 with LayerNorm, ReLU, dropout 0.1
Retrieval	k = 16 (heads), k = 4 with temperature 0.1 (k-NN); distance as in Section 4.1 with exact duration match
RAID v8 gate	MLP with 256 hidden units on [state, confidence vector]; corruption probability 0.25
Selection loss	weight 0.1, temperature 1.0; candidate action jitter 0.05 during training
Source training	90 tasks; 4,000 updates; 8 tasks × 16 windows per batch; banks of 2/5/10 demonstrations; best validation checkpoint every 250 updates; seeds 42, 43, 44
Head adaptation	batch 64; AdamW, weight decay 10^{-4}, gradient clipping 1.0; checkpoints 0, 10, 25, 50, 100, 200
Encoder tuning	MAE only, seed 42; LoRA rank 8, α 16 on blocks 10–11; head learning rate $3 \cdot 10^{-4}$; checkpoints 0, 25, 50, 100, 200
REGENT-style blend	weight $\exp(-10\,d)$ on the nearest support window, rest from the source head
Closed loop	20 query episodes per task; alignment window 24 frames; 10 settle steps; limit 1.5 × demonstration length + 20 steps; rounded gripper

B Development Results

Table B1. Development tasks (LIBERO-Goal, 10 tasks × 3 seeds), no target training: chunk MSE. The variant with the lowest MAE error was selected for the final run.

Method	MAE	DINOv2	SigLIP
VINN-style k-NN	0.592	0.471	0.506
REGENT-style blend	0.633	0.511	0.530
RAID v7	0.637	0.667	0.578
v8 gate without corruption	0.757	0.771	0.750
v8 gate with corruption (selected)	0.494	0.437	0.443

Table B2. Development tasks, after 25 updates at the selected learning rates: chunk MSE.

Method	MAE	DINOv2	SigLIP
Head-only fine-tuning	0.428	0.400	0.437
RAID v7 head	0.401	0.370	0.399
v8 gate without corruption	0.397	0.368	0.397
v8 gate with corruption (selected)	0.412	0.375	0.405

Development closed loop (raw gripper commands, before the rounding rule, MAE, 20 episodes × 10 tasks): k-NN 0.72, RAID v8 after 25 updates 0.57, LoRA after 200 updates 0.56, head-only after 25 updates 0.42, RAID v8 without training 0.34, RAID v7 without training 0.09, v8 gate without corruption 0.07.

C Run Record

The development stage ran four times. The first attempt found no free GPU; the second stopped while the pod booted; in the third, all stages except the closed loop finished, but the results were deleted with the cloud storage before they were collected (only the stage logs survived, and their numbers match the fourth run); the fourth run is the one reported. The final run ran once after one failed safety check of the cloud launcher; no final file was read before that run, and nothing was changed after it.